

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks:25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I Sakthi Priya J 192411110 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sakthi Priya J

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode– execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.
4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem Solving Approach	20	Logical, well structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Answers

3.8085 Microprocessor – Addressing Modes in an Industrial Temperature Control Unit

Introduction

An **8085 microprocessor** is an 8-bit processor widely used in embedded systems for industrial automation, monitoring, and control. In a **temperature control unit**, the microprocessor receives temperature values from sensors, processes the data, compares it with a preset value, and controls actuators such as heaters or cooling fans.

If the program uses an incorrect **addressing mode**, the processor may read the wrong memory location or incorrect sensor value, resulting in inaccurate temperature control.

Application

Industrial Temperature Control System

Applications include:

- Chemical industries
- Food processing plants
- Boiler temperature monitoring
- Cold storage systems
- Smart HVAC systems

Working:

1. Temperature sensor measures temperature.
2. Sensor data is stored in memory/I/O port.
3. 8085 reads the data.
4. It compares the temperature with the preset limit.
5. Heater or cooling fan is switched ON/OFF.

Addressing Modes and Their Influence on Data Access

Addressing Mode	Description	Example	Influence
Immediate	Data is given directly in instruction	MVI A,32H	Fast access, no memory lookup
Register	Data stored in register	MOV A,B	Very fast, used for calculations
Direct	Address is specified in instruction	LDA 2050H	Reads exact sensor memory location
Register Indirect	Register pair stores address	MOV A,M	Flexible access to sensor arrays

Addressing Mode	Description	Example	Influence
Implied	Operand is implied	CMA	No operand needed

Possible Causes of Incorrect Temperature Readings

1. Wrong Direct Address

Example:

LDA 2051H

instead of

LDA 2050H

The processor reads another memory location instead of the sensor value.

2. Incorrect Register Pair

If HL points to the wrong address,

MOV A,M

reads invalid data.

3. Immediate Addressing Used Incorrectly

Using

MVI A,30H

loads constant **30H** instead of reading the actual sensor value.

4. Register Overwritten

If another instruction changes register contents before comparison,

MOV B,A

may store incorrect data.

5. Memory Corruption

Sensor values may be overwritten accidentally due to programming mistakes.

Using the 8085 Instruction Set for Reliable Operation

Use the following instructions carefully:

Instruction	Purpose
LDA	Read sensor data from memory
STA	Store processed data
MOV	Transfer data between registers

Instruction	Purpose
MVI	Load constants
CMP	Compare temperature with limit
JC	Jump if below limit
JZ	Jump if equal
JMP	Continue program
IN	Read sensor through input port
OUT	Control heater or fan

Good programming practices:

- Use correct memory addresses.
- Initialize registers before use.
- Verify sensor data before processing.
- Use proper branching instructions.
- Keep sensor values protected in memory.

Example

Assume

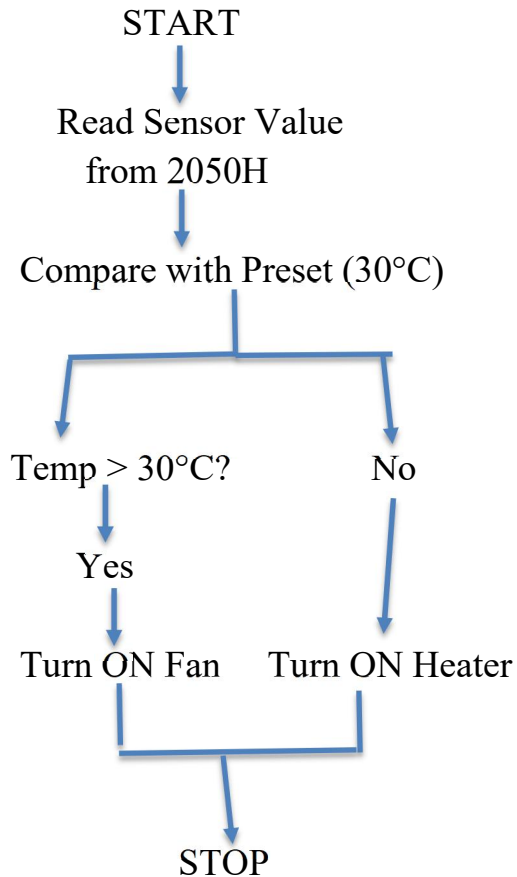
Sensor temperature stored at **2050H**

Preset temperature = **30°C (1EH)**

Algorithm

1. Read temperature.
2. Compare with 30°C.
3. If temperature is greater than 30°C
 - Turn ON cooling fan.
4. Else
 - Turn ON heater.

Flowchart



8085 Assembly Logic

assembly

LDA 2050H

CPI 1EH

JC HEATER

MVI A,01H

OUT 01H

JMP STOP

HEATER:

MVI A,00H

OUT 01H

STOP:

HLT

Equivalent C Program

```
#include <stdio.h>
```

```
int main() {  
    int temperature = 35;  
    int preset = 30;
```

```
if (temperature > preset)
    printf("Cooling Fan ON\n");
else
    printf("Heater ON\n");

return 0;
}
```

Output

If

temperature = 35

Output:

Cooling Fan ON

If

temperature = 25

Output:

Heater ON

Conclusion

The **addressing modes of the 8085 microprocessor** determine how data is accessed from registers, memory, or immediate values. In an industrial temperature control system, selecting the correct addressing mode ensures that the processor reads the correct sensor data and controls the actuators accurately. Incorrect addressing can lead to wrong temperature readings and improper operation of heaters or cooling fans. By using appropriate addressing modes along with instructions such as **LDA, MOV, CPI, JC, IN, and OUT**, the system becomes accurate, reliable, and efficient for industrial temperature monitoring and control.

4.8086 Microprocessor – Segmentation and Addressing Modes in a Multitasking Computing System

Introduction

The **8086 microprocessor** is a 16-bit processor that uses **segmented memory architecture** to access up to **1 MB of memory**. Instead of using a single memory space, it divides memory into **Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES)**. This segmentation allows multiple programs and data to coexist efficiently.

In a multitasking system, improper handling of segments can cause **incorrect data access, stack overflow, and program execution errors**. Therefore, correct use of segmentation, addressing modes, and instruction execution is essential for reliable operation.

Application

Multitasking Computing System

Applications include:

- Multi-user operating systems
- Banking software
- Industrial automation systems
- Embedded control systems
- Real-time monitoring systems

Working:

1. Program instructions are stored in the **Code Segment (CS)**.
2. Variables and data are stored in the **Data Segment (DS)**.
3. Function calls and return addresses are stored in the **Stack Segment (SS)**.
4. The CPU fetches instructions, processes data, and manages the stack during execution.

8086 Memory Segmentation

Segment	Register	Purpose
Code Segment	CS	Stores program instructions
Data Segment	DS	Stores variables and data
Stack Segment	SS	Stores stack, return addresses, local variables
Extra Segment ES		Used for additional data storage

Physical Address Calculation

Physical Address = (Segment × 10H) + Offset

Example:

- CS = 2000H
- IP = 1500H

Physical Address:

$$2000H \times 10H = 20000H$$

$$20000H + 1500H = 21500H$$

How Improper Segment Handling Causes Problems

1. Incorrect Data Access

If the **DS register** points to the wrong memory segment, the processor reads incorrect data.

Example:

assembly

```
MOV AX,[1000H]
```

If DS is incorrect, the wrong memory location is accessed.

2. Stack Overflow

Continuous **PUSH** operations without corresponding **POP** operations fill the stack memory, causing **stack overflow**.

Example:

assembly

PUSH AX

PUSH BX

PUSH CX

Without POP instructions, the stack grows beyond its allocated space.

3. Wrong Code Execution

If the **CS register** contains an incorrect segment address, the CPU fetches invalid instructions, resulting in unexpected program behavior or crashes.

4. Improper Stack Segment

If **SS** is not initialized correctly, function calls (**CALL**) and returns (**RET**) fail because return addresses are stored in the wrong location.

Addressing Modes in 8086

Addressing Mode	Description	Example
Immediate	Data is part of instruction	MOV AX,1234H
Register	Data stored in register	MOV AX,BX
Direct	Memory address given directly	MOV AX,[2000H]
Register Indirect	Register holds memory address	MOV AX,[BX]
Based	Base register used	MOV AX,[BP+04H]
Indexed	Index register used	MOV AX,[SI]
Based Indexed	Base + Index registers	MOV AX,[BX+SI]

Managing Instruction Execution Correctly

To ensure reliable program execution:

- Initialize **CS**, **DS**, and **SS** properly.
- Use correct addressing modes for accessing memory.
- Balance every **PUSH** instruction with a corresponding **POP**.
- Verify segment registers before accessing memory.
- Use **CALL** and **RET** correctly to maintain stack integrity.

Commonly used instructions:

Instruction	Purpose
MOV	Transfer data
PUSH	Store data on stack
POP	Retrieve data from stack
CALL	Call a subroutine
RET	Return from subroutine
JMP	Jump to another instruction
LEA	Load effective address

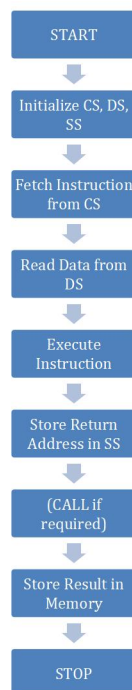
Example

Suppose:

- Code Segment stores program instructions.
- Data Segment stores two numbers.
- Stack Segment stores return addresses during function calls.

The processor reads two numbers, adds them, and stores the result.

Flowchart



8086 Assembly Program

assembly

```
MOV AX,DATA
```

```
MOV DS,AX
```

```
MOV AX,[1000H]
```

```
ADD AX,[1002H]
```

```
MOV [1004H],AX
```

```
HLT
```

Equivalent C Program

```
#include <stdio.h>
```

```
int main() {
```

```
    int num1 = 20;
```

```
    int num2 = 15;
```

```
    int result;
```

```
    result = num1 + num2;
```

```
    printf("Sum = %d\n", result);
```

```
    return 0;
```

```
}
```

Output

Sum = 35

Conclusion

The **8086 microprocessor** uses **memory segmentation** to organize program instructions, data, and stack into separate segments, allowing efficient execution of multiple programs. Incorrect handling of **Code Segment (CS)**, **Data Segment (DS)**, or **Stack Segment (SS)** can lead to wrong data access, invalid instruction execution, and stack overflow errors. By properly initializing segment registers, using suitable addressing modes, and correctly managing instructions such as **MOV**, **PUSH**, **POP**, **CALL**, and **RET**, reliable and error-free program execution can be achieved.

1.Computer Organization – CPU, Memory, I/O Interaction and Bus Structure in a Smart City Surveillance System

Introduction

A **smart city surveillance system** continuously captures video from multiple cameras and processes it for **object detection, traffic monitoring, and threat identification**. The **CPU, memory, and I/O devices** work together to transfer and process large amounts of data. During peak hours, simultaneous video streams increase the workload, causing **lag, memory access delays, and slow threat detection**. Efficient **bus structures** and an optimized **instruction execution cycle** help improve overall system performance.

Application

Smart City Surveillance System

Applications include:

- Traffic monitoring
- Crime detection
- Face recognition
- Parking management
- Public safety monitoring
- Industrial security surveillance

Working:

1. Cameras capture continuous video.
2. I/O devices transfer video data to memory.
3. CPU fetches video frames from memory.
4. CPU processes the frames to detect threats.
5. Results are displayed or alarms are generated.

Interaction Between CPU, Memory, and I/O

Component	Function
-----------	----------

CPU	Processes video data and detects threats
-----	--

Memory (RAM)	Temporarily stores video frames and program data
--------------	--

I/O Devices	Cameras, storage devices, network interfaces
-------------	--

Data Flow

Camera → I/O Device → Memory → CPU → Memory → Display / Alarm

Problem During Peak Hours

- Multiple cameras send data simultaneously.
- CPU waits for data from memory.
- Memory becomes busy handling many requests.
- I/O devices compete to transfer data.
- This results in delayed threat detection and reduced system performance.

Bus Structures

1. Single-Bus Structure

- CPU, memory, and I/O devices share the same bus.
- Only one data transfer occurs at a time.
- Creates bus congestion when many devices access the bus.

Advantages

- Simple design
- Low cost

Disadvantages

- Slow performance
- Bus bottleneck
- High waiting time

2. Multi-Bus Structure

- Separate buses are provided for CPU, memory, and I/O.
- Multiple transfers occur simultaneously.
- Reduces congestion and improves speed.

Advantages

- Faster data transfer
- Reduced CPU waiting time
- Better performance for large data processing

Disadvantages

- Higher cost
- More complex hardware

Comparison of Bus Structures

Feature	Single Bus	Multi Bus
Number of Buses	One	Multiple
Data Transfer	One at a time	Simultaneous
Speed	Low	High
Bus Congestion	High	Low
Cost	Low	High
Performance	Moderate	Excellent

Improving the Instruction Execution Cycle

The CPU executes instructions in four stages:

1. **Fetch** – Retrieve instruction from memory.
2. **Decode** – Interpret the instruction.
3. **Execute** – Perform the required operation.
4. **Store** – Save the result back to memory.

Performance Improvements

- Use **instruction pipelining** to overlap instruction stages.
- Increase **cache memory** to reduce memory access time.
- Use **Direct Memory Access (DMA)** for high-speed I/O transfers.
- Implement **interrupts** instead of continuous polling.
- Use **multi-bus architecture** to reduce CPU waiting time.

These improvements reduce lag and increase threat detection speed.

Example

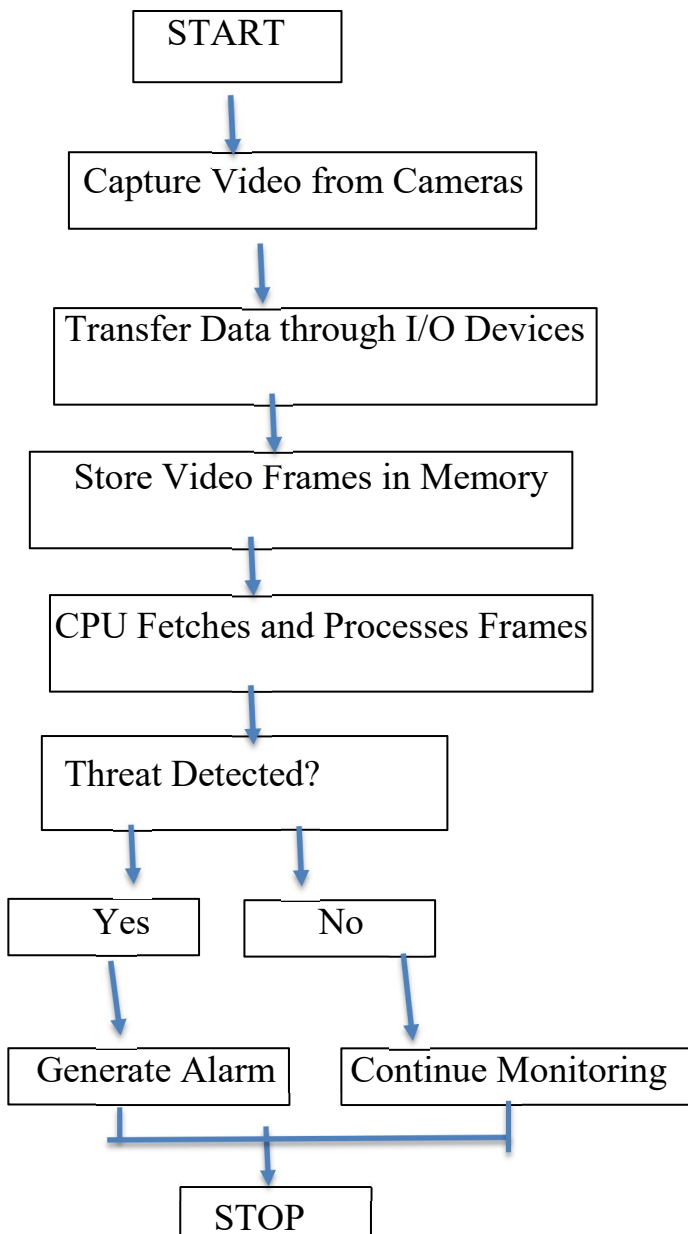
Suppose:

- 20 surveillance cameras continuously send video.
- Each camera transfers frames to RAM.
- CPU processes each frame for object detection.

Using a **single bus**, all cameras compete for the same communication path, causing delays.

Using a **multi-bus system**, cameras, memory, and CPU communicate simultaneously, improving response time and reducing lag.

Flowchart



C Program (Simulation)

```
#include <stdio.h>
```

```
int main() {  
    int cameras = 20;  
    int frames = 500;  
  
    printf("Number of Cameras : %d\n", cameras);  
    printf("Frames Processed : %d\n", frames);  
    printf("Threat Detection Completed Successfully\n");  
  
    return 0;  
}
```

Output

Number of Cameras : 20

Frames Processed : 500

Threat Detection Completed Successfully

Conclusion

A **smart city surveillance system** depends on efficient interaction between the **CPU, memory, and I/O devices** to process continuous video streams. During peak hours, a **single-bus architecture** can become a bottleneck due to bus contention, resulting in lag and delayed threat detection. A **multi-bus architecture**, along with improvements such as **instruction pipelining, cache memory, DMA, and interrupt-based I/O**, significantly enhances system performance by reducing waiting time, increasing data transfer speed, and ensuring faster and more reliable threat detection.